

```

1 -- Task 1: Total views by source (brand)
2
3 -- Select the source name and the number of views for each source
4 SELECT
5     -- Classify external_id prefixes to readable source names
6     CASE
7         WHEN c.external_id LIKE 'PGA%' THEN 'AJA'           -- If it starts with PGA → Al Jazeera Arabic
8         WHEN c.external_id LIKE 'PGD%' THEN 'AJD'           -- If it starts with PGD → Al Jazeera Documentary
9         WHEN c.external_id LIKE 'PGV%' THEN 'Atheer'         -- If it starts with PGV → Atheer (Podcasts)
10        WHEN c.external_id LIKE 'PGL%' THEN 'LIVE'           -- If it starts with PGL → Live channels
11        WHEN c.external_id LIKE 'PGC%' THEN 'Originals'       -- If it starts with PGC → Originals / Commissions
12        ELSE 'Other'                                           -- Everything else or missing external_id
13    END AS source, -- name it as source
14
15    -- Count view units for each source
16    -- A unique view = 1 user watching 1 video on 1 day.
17    -- So if the same user watches the same video 3 times in the same day → count = 1.
18    -- We build this by combining CUSTOMER_EXID + CONTENT_ID + the date part of STARTED_AT (e.g
    user123-850104-9/3/2025)
19    COUNT(DISTINCT v.CUSTOMER_EXID || '-' || v.CONTENT_ID || '-' || substr(v.STARTED_AT, 1,
    instr(v.STARTED_AT, ' ')-1)) AS unique_views
20
21 -- Data comes from the viewership table (all the watch events)
22 FROM viewership v
23
24 -- Join with the catalog table so each CONTENT_ID can be linked to its external_id
25 LEFT JOIN catalog c
26     ON v.CONTENT_ID = c.vod_dve_id
27     -- LEFT JOIN ensures we still keep the view even if the catalog row is missing (it becomes 'Other')
28
29 -- Group results so we get one row per source
30 GROUP BY source
31
32 -- Order results so the brands with the most views appear first
33 ORDER BY unique_views DESC;

```